

Phase-7 Structural Quant Engine

Workflow & Context-Efficiency Strategy Evaluation

Evaluation report only. No engine source, test, or documentation file was read, written, or modified to produce this report, and none is proposed to be changed here. The workflow strategy under review — context scoping, prompt-cache discipline, reasoning-depth tiering, and session lifecycle — is assessed specifically against Phase-7's own architecture: a multi-timeframe decision engine with a weighted-blend bias model, a state-machine risk gate, a golden-snapshot regression harness, a platform-sensitive test matrix, and an external independent-review process that gates releases.

Prepared for: Viktor · Lead Workflow and Token Optimization Engineer role, Phase-7 Structural Quant Engine

Scope constraint honored throughout: evaluation only — no code, no project files, no proposed changes implemented.

Confirmation of constraints

Understood and applied: this document does not modify engine code, test files, build scripts, or any file under `D:\phase7_engine`. It does not implement any of the workflow changes it recommends. It is an evaluation, delivered as a standalone PDF, of the token/context/workflow strategy as it applies to Phase-7 specifically.

1. Strengths of the proposed workflow

The strategy's core instinct — scope context to what a change actually touches, and reserve full architectural analysis for changes that can break something — matches how Phase-7 is already being worked, not just how it should be worked in the abstract.

- **The precision-sourcing order (functions → callers/callees → interfaces → tests → docs → broader repo) mirrors the patch-delivery discipline already in force:** a change to `bias_engine.py`'s weighting does not require re-reading `panel_render.py` unless the panel consumes the changed field, and the existing workflow already re-stages only the base files a diff touches rather than the whole tree.
- **Reasoning-depth tiering has a natural fault line in this codebase already.** Renaming `qwen_reasoning_*.txt` is lightweight; touching `decision_model.py`'s sign/label handling (Kimi Finding 2) is not. The project's own history shows what happens when that line is misjudged — Finding 3 went unscheduled for a week because a document read was treated as lightweight when it needed full-analysis depth.
- **Stable-prefix discipline is close to free here** because the project already separates stable and dynamic material by convention: the Engineering Constitution, the working rules ('one command per box', -F for commit messages, CRLF handling), and the patch-delivery skill are all long-lived and rarely edited, while the current defect, the current test output, and the current session's decisions change every turn. That is already the stable/dynamic split the strategy asks for.
- **The 'no guessing' rule matches a lesson already paid for.** Three of the five self-corrected errors on 3 September were confident claims made without checking the project's own record first. A hard rule against inferring unread implementation behavior is not new policy here — it is the existing lesson formalized.

2. Weaknesses and risks

- **"Smallest sufficient context" is judged by the strategy itself with no external check, and this project has a specific failure mode for exactly that: reading a file and treating it as read completely when only a slice was scanned.** The GLM F-7 finding and the Kimi Finding 5 sweep both turned on defaults buried in function bodies that a caller-level read would not have surfaced. A scoping rule with no tripwire will reproduce that failure under a different name.
- **The strategy has no explicit rule for cross-cutting invariants** — `code_hash` excluding `tests/` and `docs/` by directory, the golden snapshot's field set, the three-configuration test matrix, the CRLF/LF split. These are exactly the kind of fact that lives outside any single function's 'direct callers/callees' scope and has already caused two dated failures (the log-directory guard, the LF-checkout hash mismatch) when reasoned about locally instead of as a project-wide contract.
- **'Full analysis' is not itself specified with enough precision to be checked.** The category list (architecture, DecisionModel, signal integration, risk, multi-timeframe, cross-module) is right but a category list is not a procedure. Without the concrete gate this project already uses in practice — read every touched function's full body, its direct dependents, and the test asserting its contract, then diff

behaviorally-failing tests from import-failing ones — 'full analysis' can silently degrade into 'read a bit more than usual.'

- **No mechanism distinguishes context that is stale from context that is merely old.** The stable/dynamic split treats architectural principles as durably stable, but this project's own Constitution has had two amendments proposed and its head block has gone stale for a full day at least once. A caching strategy needs an invalidation trigger, not just a two-bucket split.
- **The report format itself (12 numbered sections, generic headers) is the kind of structure that reads as thorough without being checked against a specific failure.** Every claim in this document is cross-referenced against a dated Phase-7 event for that reason; a workflow document that cannot point to a specific incident it would have prevented is unverified advice wearing a report's shape.

3. Potential token-saving mechanisms

- **Diff-first delivery, already adopted.** The patch-delivery workflow never re-sends a full file when a targeted diff proves the change; extending that same discipline to analysis replies (show the changed function, not the file it lives in) is the same saving applied one step earlier.
- **Test-output triage before it enters context.** A 412-test pytest run's full console output is not information; the failing test names, the pass/fail/skip counts, and the specific assertion diffs are. The existing habit of citing 'suite 412 passing, 0 failed' rather than pasting the run is the mechanism — worth stating as a standing rule so it survives a session boundary rather than being re-derived.
- **Reference PHASE7_NEXT.md by section, not by re-reading start to finish.** At 232 KB the file is the single largest recurring context cost in this project. The existing pattern of naming exact sections to read (as was done this session) is already the fix; the risk is only that a future session reads it end to end out of habit instead of by section map.
- **Fold the recurring baseline numbers into one canonical line instead of restating them.** Suite counts, code_hash, and error counts appear dozens of times across the document's history. A single current-baseline line at the top of the head block (which already exists) removes the need to hunt for or restate the same three numbers in every commit message and every reply.
- **Route large structured evidence (manifest verification, test matrices) into tables instead of prose paragraphs.** A table encodes the same facts in roughly half the tokens of the equivalent sentence-per-figure prose this document currently uses in places.

4. Potential prompt-caching improvements

Phase-7's own material already sorts cleanly into a stable layer and a per-task layer; the improvement available is making that split explicit and byte-stable rather than re-deriving it each session.

- **Stable, rarely-edited, cache-worth-keeping:** the Engineering Constitution, the working rules block (command formatting, commit-message delivery, verification steps), the patch-delivery skill, and the architectural facts that do not change week to week — the six-factor bias blend and its weights, the $\pm 20/\pm 30$ state-machine thresholds, the code_hash exclusion rule.
- **Dynamic, changes every session:** the current suite counts, the current code_hash, the open decisions list, the current head block of PHASE7_NEXT.md.
- **The head block itself is a self-inflicted cache invalidator.** It is rewritten whole at every session boundary and sits at the very top of the file that gets read first, so any earlier-cached read of it is dead the moment it changes — which is correct, since the block is dynamic by nature, but it means the file's stable

content (rules, working practice, the sweep of items) should not be re-fetched together with the head block in the same read when only the head block needs a fresh look.

- **Do not invalidate the stable layer to fix a dynamic fact.** The Constitution does not need re-reading because the suite count changed; keep the two on separate read paths so a cache hit on the Constitution survives a routine test run.

5. Context retrieval strategy

A concrete decision tree, built from how retrieval has actually gone right and wrong on this project so far:

Question type	Minimum context	Escalate to
Mechanical / formatting change (rename a file, fix a docstring)	The file itself.	Nothing further, unless the name is cited elsewhere (the <code>qwen_reasoning_*</code> case, where a rename breaks a document that names the old files).
Single-function bug fix with a known reproduction	The function, its direct callers, the test(s) asserting its contract.	If the function's output feeds the decision path or a golden snapshot, escalate to full analysis regardless of the fix's apparent size.
New defect surfaced by running the engine, not by review	The full call path from the panel line back to the value's source — this project has repeatedly found the true defect one or two frames further back than the symptom.	
Anything touching <code>decision_model.py</code> , <code>bias_engine.py</code> 's weighting, or risk-regime classification	Full analysis by default: the function, every caller, the state machine it feeds, the golden snapshot fields it could move, and the Constitution clause it must not violate.	N/A — already at maximum scope.
Documentation / audit question	The named section of <code>PHASE7_NEXT.md</code> by line range, not the whole file.	Full end-to-end document read only when the question is itself "is the document set still accurate" — the class of question that found Finding 3.

6. Session/context lifecycle strategy

- **Session boundary = handover check, unconditionally.** This project already runs a six-question handover check before signing off; a seventh question (does `git status --short` show anything in the INDEX column) is already identified and owed. Context lifecycle and session hygiene are the same problem here: a session that ends without the handover check is a session whose context is about to be discarded with unrecorded state in it.
- **Fresh session per phase of work, not per token count.** The three-item split (Engineering Notes / semantic audit / fourteen-item sweep) is already phase-shaped: each is read-mostly or write-mostly, low-risk or decision-path, and a session specializing in one keeps its cached stable layer relevant throughout rather than alternating between doc-publishing context and decision-path context.

- **The head block of PHASE7_NEXT.md is the durable session-boundary artifact — treat it as the source of truth a new session loads, not chat history.** This is already the stated practice; the risk is only the observed failure mode (the block going stale for a day) rather than the practice itself.
- **Do not compact away the open-decisions list or the sweep's item-by-item source attribution.** Both are exactly the kind of information whose absence would force expensive re-derivation (re-reading six source documents to reassemble the sweep, or re-litigating a decision already narrowed to a specific verified band).

7. Risks of excessive context reduction

Phase-7's own record supplies concrete cases of what under-scoped context has already cost — these are not hypothetical risks.

- **Missing a finding entirely.** Finding 3, a whole Critical, went unscheduled for roughly a week because no one read the audit report end to end; a docs-are-low-priority heuristic applied here would have kept missing it indefinitely.
- **Confusing an import failure for a behavioral pass/fail.** Test evidence read without triage ('5 failed, 1 passed' with no further detail) would have overstated a defect that was really 3 collection errors plus 2 real behavioral failures — this project's rule to split failures by kind exists because that distinction was missed once.
- **Trusting one platform's green run for the other platform's claim.** The log-directory guard passed in the Linux sandbox and failed on Viktor's Windows machine at the first real run, purely because 'reduce redundant verification' would have treated a second-platform check as the kind of repetition worth cutting.
- **Treating a stable-looking number as still stable.** `code_hash` is supposed to be invariant across a docs-only change; the discipline of computing it on both trees rather than assuming exists precisely because 'this shouldn't have changed' is a claim, not a saved token.
- **Silently dropping the mechanical half of a prediction.** The stated house rule — name every consequence of a change, including the boring ones — exists because an unpredicted line in a diff is the one thing that is supposed to stop the work, and it cannot do that job if the prediction was trimmed for length.

8. Recommended operating rules for Claude

- State context scope out loud before a non-trivial change: which functions, which callers, which tests, and why that set is sufficient — one line, not a preamble.
- Never call a claim verified without saying what was read to verify it, and never call something unverified without saying so in the same sentence as the claim.
- Split test failures by kind (import/collection vs. behavioral) before reporting a count; an unsplit failure count is not evidence of anything specific.
- Predict every consequence of a change, including mechanical ones, before it runs — this is a correctness control, not a courtesy, and it stays regardless of length budget.
- Name which platform a result is evidence about, every time a sandbox result is reported as though it settles a Windows question.
- Treat `code_hash`, the golden snapshot, and the decision log's integrity as project-wide invariants that get checked on every change, not as local concerns scoped away by a small diff.

- Run the standing handover check at every session boundary regardless of how the session ends, and report what it found rather than that it ran.
- When a scoping decision is genuinely ambiguous between lightweight and full analysis, default to full analysis — this project's cost of under-scoping (a missed Critical, a Windows-only failure) has been higher than the cost of an unnecessary deep read.

9. A proposed ideal workflow for future Phase-7 tasks

- **Open with the head block of PHASE7_NEXT .md only**, then jump to the specific sections a task needs by the line-range map, exactly as this session did — never a cold full-file read unless the task is itself a document audit.
- **Classify the task before touching code:** mechanical, single-function, engine-observed defect, decision-path, or documentation. The classification sets the context floor from the table in Section 5.
- **For any decision-path change: read the function, its full dependency chain into the DecisionModel or state machine, the golden snapshot fields it could move, and the relevant Constitution clause — before writing a line of diff.**
- **Build the patch in the dedicated sandbox per the patch-delivery skill:** clone, re-stage base files off Viktor's disk immediately before diffing, run all three test configurations, verify byte-for-byte against a pristine apply.
- **Predict, then run, then compare** — golden snapshot fields, code_hash movement, suite counts, all stated before execution and checked against it after.
- **Deliver as the skill specifies:** patch file plus commit-message file, one command per box, numbered once and never renumbered, commit message carrying what was checked and what was deliberately left unfixed.
- **Close with the handover check**, updating the head block the same session the work lands, not the next one.

10. Specific recommendations for protecting architectural reasoning

- **Maintain one durable, explicit map of cross-cutting invariants** — what `code_hash` excludes and why, the golden snapshot's full field list, the three test configurations and what each catches that the others don't, the CRLF/LF split and which operations depend on it. Treat this map as part of the stable cache layer, checked against reality periodically rather than trusted indefinitely.
- **Never let a token-saving heuristic override the multi-timeframe distinction.** AERO-side and BTC-side values look structurally identical (both pass through bias/regime functions) but feed different roles in the decision, and at least one live-run finding (the false-sentence band in Kimi Finding 2) came from a function that conflated a sign check with a label check across exactly that boundary. Any scoped read touching either side should explicitly check whether the other side's semantics apply.
- **Preserve the distinction between a formal contract and what the code returns** — `StructureAnalysisResult` is already stale against `analyze()`'s real return shape. A context strategy that reads only the declared type without diffing it against the actual return will keep missing this class of drift.
- **Route every risk/decision-path change through the Constitution's clauses explicitly**, not through 'does this look risky.' Item 14's independence question is a reading of a specific clause against a specific code comment, not a judgment call a shortened context window could make safely.
- **Keep the independent-review ledger (which reviewer is clean on which family, GLM vs. Kimi/Z.ai) as durable context whenever preparing or reading an audit round** — it is exactly the kind of cross-session fact that a token-saving pass would discard as 'not code' and that turned out to change what a report could be trusted to mean.

11. Safe vs. dangerous token optimization

Safe (removes redundancy only)	Dangerous (removes information needed for correct reasoning)
Citing suite counts instead of pasting full pytest output.	Reporting a failure count without splitting import errors from behavioral failures.
Reading a named section of PHASE7_NEXT.md instead of the whole file for a scoped question.	Skipping the end-to-end document read when the question is whether the document set itself is still accurate (the Finding-3-shaped question).
Showing a diff instead of a full file for a targeted change.	Omitting the mechanical, uninteresting consequences from a change prediction.
Reusing a cached, byte-stable Constitution / working-rules block across turns.	Reusing a cached read of the head block after it has been rewritten.
Skipping a second read of a function already fully read this session.	Skipping the second platform's verification because the first platform's run was green.
Summarizing a long independent-review report's overlap/divergence instead of quoting it in full.	Dropping which reviewer produced which finding, when independence-of-source is itself part of what the finding is worth.
Batching related tool calls / file reads that don't depend on each other's results.	Assuming a function's behavior instead of reading it, to avoid the read.

The dividing line in every row is the same one: safe optimization removes a second copy of information already present; dangerous optimization removes the one copy that carries the distinction a later decision depends on.

12. Concise final recommended workflow

- Load the head block only. Classify the task.
- Pull the minimum context the classification requires (Section 5's table); escalate to full analysis for anything touching the decision path, risk, or multi-timeframe logic.
- State scope, assumptions, and what remains unverified — in the same sentence as any claim, not as a caveat after it.
- For code changes: sandbox per the patch-delivery skill, predict before running (golden snapshot, code_hash, suite counts, every mechanical consequence), then verify byte-for-byte against a pristine apply in all three test configurations.
- Report results split by kind and by platform; never let a green Linux run stand in for a Windows claim.
- Deliver via the established mechanism (patch + commit-message file, one command per box), run the handover check, and update the head block the same session.

This report proposes the above; nothing in it has been applied. Adopting any of it is a workflow ruling, not a code change, and is Viktor's call like the other open decisions.